Uncrossing Wires for 3-SAT: A Potential That Closes the Recursion

Siyuan Zhuang

October 31, 2026

Abstract

We prove that for each $\varepsilon > 0$, 3-SAT with n variables can be decided by a deterministic algorithm in time $C_\varepsilon |\varphi|^{O_\varepsilon(1)} 2^{\varepsilon n}$, where $|\varphi|$ is the size of the input. So the Exponential Time Hypothesis fails. OpenAI and Claude (Anthropic) first announced this statement. Our proof has these features. Besides the branching that reduces the occurrences of a variable, it is elementary: one identity on four bits, a count of strings, and the Cauchy–Schwarz inequality. It has no polynomial approximation, no spectral argument and no determinant. It is a recursion on one fixed tower of grids, with two pieces for each rectangle, and with no regrouping of nodes.

The main finding is a measure that closes the recursion. View a formula as a network: nodes on a grid, joined by wires that each hold a bit. The number of wires need not fall from one level of the recursion to the next: a rectangle with wires on only one diagonal keeps all of them. But those wires then occupy fewer pairs of nodes, and a quadratic potential sees this. It falls by the fixed factor $\sqrt{24/25}$ for every term, in one of the two directions, whatever the weights are. So weights need not be balanced, and one short inequality closes the recursion. Two wires that cross both the rows and the columns of a grid are replaced by two terms, each crossing only one of them; a choice of terms gives a network on the rows or on the columns; and we keep the one with the smaller potential. The theorem is formalized in Lean 4 down to a multitape machine, with its time and space.

The result and the strategy

Theorem 1. *For each $\varepsilon > 0$, 3-SAT with n variables is decided deterministically in time $C_\varepsilon |\varphi|^{O_\varepsilon(1)} 2^{\varepsilon n}$, where $|\varphi|$ is the size of the input φ.*

The Exponential Time Hypothesis of Impagliazzo and Paturi [IP01] says that for some $\delta > 0$, no algorithm decides 3-SAT on n variables in time $2^{\delta n}$ times a polynomial in the input size. Theorem 1 contradicts this: take $\varepsilon < \delta$. The degree of the polynomial may depend on ε; after duplicates are removed the input has size polynomial in n.

Background and what is simplified

OpenAI [OAI26] and, independently, Claude (Anthropic) [Cla26] announced algorithms for counting with explicit subexponential rates. O'Donnell [O'D26] and Kintali [Kin26] gave expositions of OpenAI's approach, and a community examination of the argument has been called for [ETH26]. Like the proof of Theorem 2 in [Cla26] and like Kintali, we fix ε and leave the rate unoptimized. We aim for a proof that is short to state and easy to follow.

The next paragraphs use terms that the strategy below explains. A formula becomes a *network*: nodes joined by *wires*, each holding a bit. The nodes are placed on a grid of rows and columns, and two rows and two columns form a *rectangle*. The bits on a rectangle are expanded into *pieces*, and all nodes of a row, or of a column, are then *merged* into one node.

Table 1 lists how the proofs remove the wires that cross every partition. The closest route is Claude's. The grid of rows and columns, the rectangles, the expansion with one piece per rectangle, and the choice to merge rows or columns for each term are the core of Claude's recursion (Sections 2.5, 4 and 5 there). The difference is in what is tracked. The proof of Theorem 2 in [Cla26], a weaker form of its main theorem, works on cliques with a uniform bound on the bits of an edge. After a merge the edges are not uniform, and padding them would cancel the saving, so each level also regroups the merged nodes by a second-moment argument, which needs a grid of side $3K^8$ for K groups (Section 2.6 and Lemma 1 there). Here the potential Φ of Section 4 takes the place of that step. It allows arbitrary weights, so the number of nodes at each level is the square root of the last one, and no node is regrouped. Quadratic quantities do occur in the cited works as balance tools inside one level or one schedule: the variance in Claude's Lemma 1, and the potentials of Kintali's schedules. We did not find one tracked across the levels of a recursion. Also, that proof in [Cla26] applies an eight-square identity to blocks of three bits, with a number of pieces per rectangle that grows with the bits; here the one-bit crossing identity of [Cla26, Appendix H], which no proof there uses, is enough, with two pieces. That appendix also computes the gain, $\log_2 12$ of 4 bits, for pieces built from single bits, and Lemma 1 is a count of this kind. In return, Claude's route gives an explicit rate, through a changed form of its recursion (Appendices A and B there).

Work	**The wires that cross every partition**	**Recovery of the value**
OpenAI [OAI26], main text	low-degree forms, enumerated integer measurements	Hermite interpolation at nodes
OpenAI [OAI26], Appendix D	modes between groups, recursion on wires	tapered Hermite recovery
Kintali [Kin26]	low-degree forms, enumerated integer measurements	Hermite interpolation at nodes
O'Donnell [O'D26]	block matrix plus low rank, program on color counts	Hermite interpolation at nodes
Claude [Cla26]	eight-square identity, recursion on cliques with regrouping	sum over the recursion
This paper	one-bit identity, recursion on a fixed tower with a potential	sum over chains

Table 1: Where the proofs differ. For Appendix D of [OAI26] we read the overview and the headings.

The proof keeps the branching of Impagliazzo, Paturi and Zane [IPZ01]. Its placement of a formula can be compared with the balanced greedy labeling of O'Donnell [O'D26, Lemma 4.1].

The strategy

A sum over assignments would split if the clauses respected some partition of the variables into small groups, but a sparse formula can be an expander, where every such partition is crossed by many clauses. So we do not look for one partition. We expand the formula into a short sum of terms, and each term has its own order of summation.

A formula is a network: nodes joined by wires (Section 1). If few wires remain, the sum over all bits is cheap. Arrange the nodes in a grid. Two wires that join opposite corners of a rectangle cross both the rows and the columns. An exact identity replaces them by two terms: the first term compares bits only within columns, the second only within rows. A rectangle can carry many such pairs of wires, and one identity for each pair would give a number of terms exponential in the number of wires. So we group the terms of a rectangle into two pieces: one piece needs fewer new wires between the rows, the other fewer new wires between the columns (Section 2).

Choosing one piece for each rectangle gives a sum of terms whose number depends only on the grid. Each term can be written as a network on the rows alone, or on the columns alone, by merging the nodes of a row or of a column (Section 3). A term may mix the two kinds of pieces, so the number of wires need not fall. But a quadratic potential, which bounds the number of wires, falls by a fixed factor in one of the two networks (Section 4). We keep that network and repeat, until two nodes and few wires remain (Section 5). The potential is small if each variable has few occurrences, spread evenly over the pairs of nodes (Section 6), and branching reduces every formula to few occurrences per variable (Section 7).

Throughout, a statement in square brackets stands for 1 if it holds and 0 otherwise, and $[K] = \{0, \dots, K-1\}$. *Cauchy* means $(\sum_{i\in I} c_i d_i)^2 \le \sum_{i\in I} c_i^2 \sum_{i\in I} d_i^2$; with all $d_i = 1$ it reads $(\sum_{i\in I} c_i)^2 \le |I| \sum_{i\in I} c_i^2$.

1 Networks

We start with the object that the proof transforms. A *network* has *nodes* $s \in [K]$. A *weight* $w(s,t) = w(t,s) \ge 0$ says how many *wires* run between the nodes s and t. Each wire holds one bit. The bits of the wires between s and t form one number below $2^{w(s,t)}$, and each of the two nodes has a copy of it: $B(s,t)$ at the node s, and $B(t,s)$ at the node t. The value below tests that the two copies agree, and Section 2 rewrites these tests. Wires with both ends on one node are allowed, so $w(s,s)$ may be positive; they have the single number $B(s,s)$. A *node function* F_s reads the numbers $B(s,\cdot)$ at the node s, and its values are dyadic rationals, that is, fractions with a power of 2 as denominator. The *value* of the network is

$$\mathrm{val}(K,w,F) = \sum_B \prod_{s,t\in[K]} [B(s,t) = B(t,s)] \prod_{s\in[K]} F_s(B(s,\cdot)),$$

where B runs over the families of numbers with $B(s,t) < 2^{w(s,t)}$. The tests leave one number for each pair of nodes, and each node function reads its own copy. The enumeration of all these numbers costs about $2^{\#\text{wires}}$ evaluations of the F_s. Our aim is to write val as a short sum of values of networks with few wires.

Example. Let $K = 4$, and give each node a function of one bit: $F_0 = (1,2)$, $F_1 = (1,1)$, $F_2 = (2,5)$, $F_3 = (3,1)$ at the bits 0 and 1. Let $w(0,3) = w(1,2) = 1$, and let all other weights be zero. With $x = B(0,3) = B(3,0)$ and $y = B(1,2) = B(2,1)$, the value is

$$\sum_{x,y\in\{0,1\}} F_0(x)F_3(x)F_1(y)F_2(y) = (1\cdot 3 + 2\cdot 1)(1\cdot 2 + 1\cdot 5) = 35.$$

2 Uncrossing one rectangle

Let $K = K'K'$, and write a node as (a,b) with $a,b \in [K']$; its number is $aK' + b$. Then a is its row and b its column. We want to merge the nodes of a row, or of a column, into one node. Merging turns each wire inside a row into a sum inside the new node, and the same holds for a column. A wire between different rows and different columns survives both merges. In the example the nodes are $0 = (0,0)$, $1 = (0,1)$, $2 = (1,0)$, $3 = (1,1)$, and both wires join different rows and different columns.

Take rows $a \ne a'$ and columns $b \ne b'$. Let the bits x, y, u, v lie on the four nodes $(a,b), (a,b'), (a',b), (a',b')$ of this *rectangle*, in this order. The *crossing* $[x = v][y = u]$ com-

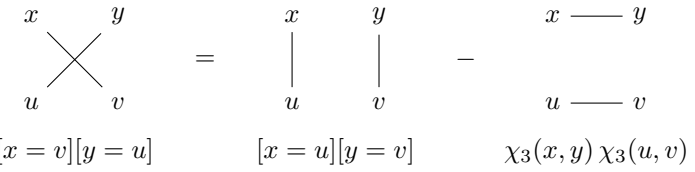
pares opposite corners. For a bit p put $\hat{p} = (-1)^p$, and let

$$\chi_0(p,q) = \tfrac{1}{2}(1+\hat{p}\hat{q}) = [p=q], \qquad \chi_3(p,q) = \tfrac{1}{2}(\hat{p}-\hat{q}) = [p\neq q]\hat{p}.$$

Put $U = \chi_0(x,u)\chi_0(y,v) = [x=u][y=v]$ and $S = \chi_3(x,y)\chi_3(u,v)$. Then

$$[x=v][y=u] = \tfrac{1}{4}(1+\hat{x}\hat{v})(1+\hat{y}\hat{u}) = \tfrac{1}{4}(1+\hat{x}\hat{u})(1+\hat{y}\hat{v}) - \tfrac{1}{4}(\hat{x}-\hat{y})(\hat{u}-\hat{v}) = U - S, \quad (1)$$

because both sides equal $\frac{1}{4}(1+\hat{x}\hat{v}+\hat{y}\hat{u}+\hat{x}\hat{y}\hat{u}\hat{v})$: on the right, the terms $\hat{x}\hat{u}$ and $\hat{y}\hat{v}$ cancel. In the figure, a line joins the two bits that one factor reads.



A product $g(x,u)g'(y,v)$ is *across the columns*: one factor reads only the column b, and the other only the column b'. A product $g(x,y)g'(u,v)$ is *across the rows*. So U is one product across the columns, and S is one product across the rows. Thus (1) replaces two wires that survive both merges by U, which needs no wire between the two columns once they are merged, and S, which needs no wire between the two rows once they are merged. In the example, multiply (1) by $F_0(x)F_1(y)F_2(u)F_3(v)$ and sum over the four bits:

$$\begin{aligned} 35 &= \Big(\sum_x F_0(x)F_2(x)\Big)\Big(\sum_y F_1(y)F_3(y)\Big) - \Big(\sum_{x,y} F_0(x)F_1(y)\chi_3(x,y)\Big)\Big(\sum_{u,v} F_2(u)F_3(v)\chi_3(u,v)\Big) \\ &= 12\cdot 4 - (-1)(-13) = 48 - 13. \end{aligned}$$

Here 48 is a product over the two columns, and 13 is a product over the two rows.

Many wires. A rectangle can carry many wires. Let x, y, u, v be numbers of m bits, with bits x_j, y_j, u_j, v_j for $0 \le j < m$, and let U_j, S_j be U, S at the position j. By (1), the crossing $[x=v][y=u]$ is $\prod_j (U_j - S_j)$. Multiplied out, this is a sum of 2^m products, one for each set T of positions that take $-S_j$. If each product became its own term, the number of terms of a network would be exponential in its number of wires. With two terms per rectangle, that number depends only on the grid. So we group the products by the size of T, into two *pieces*:

$$P^{\mathrm{r}} = \sum_{|T|\ge m/2} \prod_{j\in T}(-S_j) \prod_{j\notin T} U_j, \qquad P^{\mathrm{c}} = \sum_{|T|< m/2} \prod_{j\in T}(-S_j) \prod_{j\notin T} U_j, \qquad P^{\mathrm{r}} + P^{\mathrm{c}} = [x=v][y=u].$$

For $m = 1$ this is (1): $P^{\mathrm{r}} = -S$ and $P^{\mathrm{c}} = U$, which contribute -13 and 48 in the example.

A piece costs new wires. Suppose a piece is a sum of products across the rows, over an index ξ of d bits. In Section 3, ξ becomes d wires between the two rows, in place of the $2m$ wires of the crossing; the same holds for the columns. So our goal is $d < 2m$, for P^{r} across the rows and for P^{c} across the columns (Lemma 1). For this we expand U_j and S_j both ways. We need two more functions of two bits,

$$\chi_1(p,q) = \tfrac{1}{2}(1-\hat{p}\hat{q}) = [p\neq q], \qquad \chi_2(p,q) = \tfrac{1}{2}(\hat{p}+\hat{q}) = [p=q]\hat{p}.$$

Then, with c running over $0, 1, 2, 3$,

$$2U_j = \sum_c \chi_c(x_j, y_j)\chi_c(u_j, v_j),$$
$$2S_j = \chi_0(x_j, u_j)\chi_0(y_j, v_j) - \chi_1(x_j, u_j)\chi_1(y_j, v_j) - \chi_2(x_j, u_j)\chi_2(y_j, v_j) + \chi_3(x_j, u_j)\chi_3(y_j, v_j).$$

To check the first line, add in pairs: the terms $c = 0, 1$ give $\frac{1}{2}(1 + \hat{x}_j\hat{y}_j\hat{u}_j\hat{v}_j)$, and the terms $c = 2, 3$ give $\frac{1}{2}(\hat{x}_j\hat{u}_j + \hat{y}_j\hat{v}_j)$. In the second line, the terms $c = 0, 1$ give $\frac{1}{2}(\hat{x}_j\hat{u}_j + \hat{y}_j\hat{v}_j)$, and the terms $c = 2, 3$ give $-\frac{1}{2}(\hat{x}_j\hat{v}_j + \hat{y}_j\hat{u}_j)$. A *string* is an element π of $\{0, 1, 2, 3\}^m$. Let $f_\pi(p, q) = \prod_j \chi_{\pi_j}(p_j, q_j)$ for numbers p, q of m bits, and let $n_c(\pi)$ be the number of letters c of π.

Lemma 1. *Let $m = 0$, or let $m = 2k + 1$ be odd. Put $\beta_0 = 0$ and $\beta_{2k+1} = 2k + 1 + \lceil 8k/5 \rceil$. Then $\beta_m < 2m$ for odd m, because $\lceil 8k/5 \rceil \le 2k$. Each of $P^{\mathrm{r}}, P^{\mathrm{c}}$ is a sum over an index ξ of products $g_\xi(x, y)g'_\xi(u, v)$ (across the rows), and also a sum over an index ξ of products $g_\xi(x, u)g'_\xi(y, v)$ (across the columns). The index has this number of bits:*

	across the rows	*across the columns*
P^{r}	β_m	$2m$
P^{c}	$2m$	β_m

All values are dyadic with $O(m + 1)$ bits, and they are computed in time polynomial in m.

Proof. For $m = 0$ there is only $T = \varnothing$, so $P^{\mathrm{r}} = 1$ and $P^{\mathrm{c}} = 0$. Let $m = 2k + 1$. Fix T and multiply out the expansions at each position.

Across the rows. At $j \in T$ we have $-S_j = -\chi_3(x_j, y_j)\chi_3(u_j, v_j)$, and at $j \notin T$ we have $U_j = \frac{1}{2}\sum_c \chi_c(x_j, y_j)\chi_c(u_j, v_j)$. So the product of T is $(-1)^{|T|}2^{|T|-m}$ times the sum of $f_\pi(x, y)f_\pi(u, v)$ over the strings π with the letter 3 on all of T.

Across the columns. At $j \notin T$ we have $U_j = \chi_0(x_j, u_j)\chi_0(y_j, v_j)$, and at $j \in T$ we have $-S_j = \frac{1}{2}(-\chi_0\chi_0 + \chi_1\chi_1 + \chi_2\chi_2 - \chi_3\chi_3)$, each factor at (x_j, u_j) and (y_j, v_j). So the product of T is $2^{-|T|}$ times a signed sum of $f_\pi(x, u)f_\pi(y, v)$ over the strings π with the letter 0 off T.

So each piece is, across either cut, a sum over π of a number c_π times $f_\pi f_\pi$. For a string π, a set T that fits it contributes a sign and a power of two that depend only on $|T|$ and on π, and the number of fitting sets of size i is $\binom{n_3(\pi)}{i}$ across the rows and $\binom{n_0(\pi)}{i-m+n_0(\pi)}$ across the columns. So c_π is a sum of at most $m + 1$ numbers, each a binomial times a power of two with a sign. For $2m$ bits, the index names π with two bits per letter; then $g_\xi = c_\pi f_\pi$ and $g'_\xi = f_\pi$.

For β_m bits: a set T of P^{r} has $|T| \ge k + 1$, so across the rows only strings with at most k letters other than 3 occur. A set T of P^{c} has $|T| \le k$, so across the columns only strings with at most k letters other than 0 occur. Each of these other letters has three values. So the first m bits of the index mark the positions of the special letter (3 for P^{r}, 0 for P^{c}), and the other $\lceil 8k/5 \rceil$ bits hold a number below 3^k, whose base 3 digits are the other letters, in order. Indexes that do not arise get the zero function. The number fits because $3^{5k} = 243^k \le 256^k = 2^{8k} \le 2^{5\lceil 8k/5 \rceil}$. □

3 One step

Section 2 treats one rectangle. A network has wires on many pairs of nodes. We now apply the split to all rectangles at once and then merge nodes, so that the value becomes a sum of values of networks with K' nodes.

Let $(K'K', w, F)$ be a network. Two rows $a < a'$ and two columns $b < b'$ define a rectangle ρ. Its *diagonals* are the pairs of nodes $\{(a, b), (a', b')\}$ and $\{(a, b'), (a', b)\}$. Each pair of nodes in

different rows and different columns is a diagonal of exactly one rectangle. Let w_1, w_2 be the weights of the two diagonals of ρ, and let m_ρ be the largest odd number at most $\min(w_1, w_2)$, or 0 if the minimum is 0. (Lemma 1 takes m odd or 0. An even m would save nothing at $m = 2$: the index would have $2 + \lceil 8/5 \rceil = 4 = 2m$ bits.) The lowest m_ρ bits of the numbers on the two diagonals are the *paired* bits: with the numbers

$$x = B((a,b),(a',b')), \quad v = B((a',b'),(a,b)), \quad y = B((a,b'),(a',b)), \quad u = B((a',b),(a,b')),$$

taken modulo 2^{m_ρ}, the condition that the paired bits agree is the crossing $[x = v][y = u]$ of Section 2. A pair of nodes that is not a diagonal of a rectangle has no paired bits. All other bits of a number are its *carried* bits.

Choices. A *choice* σ picks, for each rectangle ρ, one of the two pieces $P_\rho \in \{P^{\mathrm{r}}, P^{\mathrm{c}}\}$ of its crossing. There are $(K'(K'-1)/2)^2 \le K'^4/4$ rectangles, so at most $2^{K'^4/4}$ choices. Fix B. The test $[B(s,t) = B(t,s)]$ says that the carried bits agree and the paired bits agree. Over all pairs of nodes, the tests of the paired bits multiply to $\prod_\rho [x = v][y = u] = \prod_\rho (P^{\mathrm{r}} + P^{\mathrm{c}})$, at the numbers x, y, u, v of each ρ. Multiplying out this product gives one term for each σ:

$$\prod_{s,t} [B(s,t) = B(t,s)] = \sum_\sigma \prod_{s,t} [\text{the carried bits of } B(s,t) \text{ and } B(t,s) \text{ agree}] \prod_\rho P_\rho. \tag{2}$$

Keeping the rows. Fix σ. We build a network on the K' rows. All numbers on the nodes of one row will be summed inside one new node function. So the wires inside a row disappear, and only what joins two rows must stay a wire. For a rectangle ρ, let d^{r}_ρ be β_{m_ρ} if $P_\rho = P^{\mathrm{r}}$, and $2m_\rho$ if $P_\rho = P^{\mathrm{c}}$; by Lemma 1, $P_\rho = \sum_\xi g_\xi(x,y) g'_\xi(u,v)$ with an index of d^{r}_ρ bits. For rows $a < a'$, the new wires between a and a', and what the new nodes do with them, are:

- for all columns b, b', a copy of the carried bits of the number between (a,b) and (a',b'). The node a tests that this copy equals the carried bits of $B((a,b),(a',b'))$. The node a' tests that it equals the carried bits of $B((a',b'),(a,b))$;
- for all rectangles ρ with these rows, the d^{r}_ρ bits of the index ξ of ρ. The node a has the factor $g_\xi(x,y)$, and the node a' has the factor $g'_\xi(u,v)$.

So the new weight is

$$w^{\mathrm{r}}(a,a') = \sum_b w((a,b),(a',b)) + \sum_{b<b'} f^{\mathrm{r}}_\rho, \qquad f^{\mathrm{r}}_\rho = w_1 + w_2 - 2m_\rho + d^{\mathrm{r}}_\rho, \tag{3}$$

where ρ is the rectangle with the rows a, a' and the columns b, b', and w_1, w_2 are the weights of its diagonals: f^{r}_ρ counts the carried bits on its diagonals and the bits of its index. Also $w^{\mathrm{r}}(a,a) = 0$. The new node function at the row a reads the family B' of new numbers at a, and sums over the old numbers on all nodes of the row a:

$$F^{\mathrm{r}}_a(B') = \sum_B \mathsf{T}_a \prod_b F_{(a,b)}(B((a,b),\cdot)), \tag{4}$$

where B runs over the numbers $B(s,t) < 2^{w(s,t)}$ for the nodes s of the row a and all nodes t. Here T_a is the product of the tests and factors of the node a in the list above, for all other rows, and of the tests $B(s,t) = B(t,s)$ for two nodes s, t of the row a. All of these read only numbers on the row a.

Keeping the columns. Exchanging rows and columns gives a network $(K', w^{\mathrm{c}}, F^{\mathrm{c}})$ on the columns: now the factors of a rectangle are across the columns, $g_\xi(x,u)$ and $g'_\xi(y,v)$; d^{c}_ρ is β_{m_ρ} if $P_\rho = P^{\mathrm{c}}$, and $2m_\rho$ if $P_\rho = P^{\mathrm{r}}$; and $f^{\mathrm{c}}_\rho = w_1 + w_2 - 2m_\rho + d^{\mathrm{c}}_\rho$.

Lemma 2. *For each σ, pick the network on the rows or the network on the columns. Then* $\mathrm{val}(K'K', w, F)$ *is the sum over σ of the values of the picked networks.*

Proof. Consider the rows. Insert (4) in the value of the new network, multiply out the product over a and reorder the sums. The value is the sum over all old B of $\prod_s F_s(B(s,\cdot))$ times the sum of $\prod_a T_a$ over the new numbers; the tests of the new network leave one new number for each pair $a < a'$. Fix B. The new numbers are independent for different pairs $a < a'$, and each one splits into independent parts. A part that copies carried bits has two tests, one from each of the rows a, a'. Its sum is one test: the carried bits of $B(s,t)$ and $B(t,s)$ agree. The part of the index of ρ has the sum $\sum_\xi g_\xi(x,y)g'_\xi(u,v) = P_\rho$. The tests inside a row are the tests of the carried bits for pairs in one row. So this sum is the summand of (2), and the sum over σ gives the product of all tests $[B(s,t) = B(t,s)]$. The columns are the same with the other expansion in Lemma 1. □

4 The potential

A rectangle with P^{r} has fewer new wires in the rows, and a rectangle with P^{c} has fewer in the columns. A choice may mix them, and a rectangle with $m_\rho = 0$ keeps all its wires in both networks. So the number of wires need not fall. What does change is how the wires are spread over the pairs of nodes. In the network with $K' = 2$ and d wires between $(0,0)$ and $(1,1)$ only, the new network on the rows has d wires between its two nodes. These wires occupied one pair of nodes out of six, and now they occupy the only pair. So we measure a network by a number that also sees the spread, the *potential*

$$\Phi(K,w) = K\Big(\sum_{s,t\in[K]} w(s,t)^2\Big)^{1/2}.$$

For a fixed sum of weights, Φ is smallest when all pairs of nodes have the same weight, and then Φ is about twice the number of wires. In this network, Φ falls from $\sqrt{32}\,d$ to $\sqrt{8}\,d$ while the number of wires stays d. Cauchy over the K^2 ordered pairs of nodes gives

$$\sum_{s,t\in[K]} w(s,t) \le \Phi(K,w). \tag{5}$$

The left side is the number of bits of a family B. So a small potential makes the sum over B in val short. The potential falls by a fixed factor in one of the two networks:

Lemma 3. *Let $\gamma = \sqrt{24/25}$. For every σ,* $\min\big(\Phi(K', w^{\mathrm{r}}), \Phi(K', w^{\mathrm{c}})\big) \le \gamma\,\Phi(K'K', w)$.

Proof. Fix rows $a \ne a'$, and write each f^{r}_ρ in (3) as $\frac12 f^{\mathrm{r}}_\rho + \frac12 f^{\mathrm{r}}_\rho$. Then (3) has $K' + K'(K'-1) = K'^2$ terms, and Cauchy gives

$$w^{\mathrm{r}}(a,a')^2 \le K'^2\Big(\sum_{b<b'} \tfrac12 (f^{\mathrm{r}}_\rho)^2 + \sum_b w((a,b),(a',b))^2\Big).$$

Sum over the ordered pairs $a \ne a'$ and multiply by K'^2. The left side becomes $\Phi(K', w^{\mathrm{r}})^2$. A rectangle arises from (a,a') and from (a',a), so the $\frac12$ disappears. The second sum becomes the sum over the ordered pairs of different nodes in one column. In the same way for the columns, so

$$\Phi(K', w^{\mathrm{r}})^2 + \Phi(K', w^{\mathrm{c}})^2 \le K'^4\Big(\sum_\rho \big((f^{\mathrm{r}}_\rho)^2 + (f^{\mathrm{c}}_\rho)^2\big) + \sum_{\substack{s\ne t \text{ in one row}\\ \text{or one column}}} w(s,t)^2\Big).$$

Fix ρ, with $w_1 \geq w_2$, and $m = m_\rho$. One of $d^{\mathrm{r}}_\rho, d^{\mathrm{c}}_\rho$ is $2m$, which gives $f = w_1 + w_2$. The other is β_m, which gives $f \leq w_1 + \frac{9}{10} w_2$. Indeed, if $m = 0$ then $w_2 = 0$ and $f = w_1$. If $m = 2k+1$ then $f - w_1 = w_2 - m + \lceil 8k/5 \rceil$ and $\lceil 8k/5 \rceil \leq (8k+4)/5$, so $10(f - w_1) \leq 10(w_2 - m) + 16k + 8$. Since m is the largest odd number at most w_2, there are two cases: $w_2 = 2k+1$, where this bound is $16k + 8 \leq 9w_2$; and $w_2 = 2k+2$, where it is $16k + 18 \leq 9w_2$. So Cauchy gives

$$(f^{\mathrm{r}}_\rho)^2 + (f^{\mathrm{c}}_\rho)^2 \leq (w_1 + w_2)^2 + (w_1 + \tfrac{9}{10} w_2)^2 \leq (2 + 1 + \tfrac{81}{100})(w_1^2 + w_2^2) \leq \tfrac{96}{25}(w_1^2 + w_2^2).$$

Each diagonal is two ordered pairs of nodes, so $w_1^2 + w_2^2$ is half of the sum of $w(s,t)^2$ over the four ordered pairs of ρ. So the whole right side is at most $K'^4 \cdot \frac{48}{25} \sum_{s,t} w(s,t)^2 = \frac{48}{25} \Phi(K'K', w)^2$. The smaller of the two potentials has a square at most half of this. (Without the saving of Lemma 1 the constant $\frac{96}{25}$ would be 4, and the potential would not fall.) □

5 The chain of steps, and its cost

We now iterate until two nodes remain. Let $\ell \geq 0$ and $K_j = 2^{2^{\ell - j}}$, so that $K_0 = 2^{2^\ell}$, $K_\ell = 2$ and $K_j = K_{j+1} K_{j+1}$.

Theorem 2. *Let (K_0, w, F) be a network, with potential $\Phi = \Phi(K_0, w)$. Then $\mathrm{val}(K_0, w, F)$ is computed in time $2^{K_0^2/3 + 4\gamma^\ell \Phi}$ times a constant depending on ℓ, times a polynomial in Φ and the time and the bit length of one evaluation of a node function.*

Proof. Put $N_0 = (K_0, w, F)$. Given N_j with $j < \ell$, pick a choice σ_j for N_j, and let N_{j+1} be the one of its two networks with the smaller potential, and the network on the rows if they are equal. A *chain* is a sequence $(\sigma_0, \ldots, \sigma_{\ell-1})$. By Lemma 2, $\mathrm{val}(N_0)$ is the sum of $\mathrm{val}(N_\ell)$ over all chains. By Lemma 3, N_j has potential at most $\gamma^j \Phi$.

The network N_j has at most $K_{j+1}^4/4 = K_j^2/4$ rectangles, and the next network has at most $K_{j+1}^2/4 = K_j/4$, which is at most a quarter of $K_j^2/4$, since $K_j \geq 4$. So all N_j with $j < \ell$ have at most $K_0^2/3$ rectangles together, and there are at most $2^{K_0^2/3}$ chains.

Fix a chain. Computing all weights takes polynomial time, and it fixes which of the two networks is kept, since a comparison of potentials is a comparison of two sums of squares. By (5), a family B of N_ℓ has at most $\gamma^\ell \Phi$ bits, so the value of N_ℓ is a sum over at most $2^{\gamma^\ell \Phi}$ families B.

A node function of N_{j+1} is (4): a sum over the numbers $B((a,b),t)$ on one row of N_j, or on one column. Their number of bits is $\sum_b \sum_t w((a,b),t)$, a sum of K_{j+1}^3 weights w of N_j. By Cauchy its square is at most $K_{j+1}^3 \sum_{s,t} w(s,t)^2 = \Phi(N_j)^2 / K_{j+1}$, because $\Phi(N_j)^2 = K_{j+1}^4 \sum_{s,t} w(s,t)^2$. So these numbers have at most $E_j = \gamma^j \Phi / \sqrt{K_{j+1}}$ bits. Each term of the sum has one evaluation of T and K_{j+1} evaluations of node functions of N_j. These evaluations are separate, so their costs add. So one evaluation of a node function of N_ℓ takes at most $\prod_{j<\ell} (K_{j+1} 2^{E_j})$ evaluations of node functions of N_0, and as many evaluations of T. For $r = \ell - j \geq 1$ we have $\sqrt{K_{j+1}} = 2^{2^{r-2}} \geq 2^{r-1}$, so

$$\sum_{j<\ell} E_j \leq \gamma^\ell \Phi \sum_{r \geq 1} \gamma^{-r} 2^{1-r} = \frac{2\gamma^\ell \Phi}{2\gamma - 1} \leq 3\gamma^\ell \Phi,$$

since $\gamma \geq 5/6$. Now multiply: there are at most $2^{K_0^2/3}$ chains; for one chain, the value of N_ℓ has at most $2^{\gamma^\ell \Phi}$ families and two evaluations of node functions of N_ℓ for each family; each of these takes at most $2^{3\gamma^\ell \Phi} \prod_{j<\ell} K_{j+1}$ evaluations of node functions of N_0 and of T; and $\prod_{j<\ell} K_{j+1}$ is a constant depending on ℓ. By Lemma 1, T takes polynomial time. All numbers of wires are at

most the number of wires of N_0, since $\beta_m \le 2m$, and so at most Φ. For the bits of values: if the values at level j have at most b_j bits, a value at level $j+1$ is a sum of at most 2^{E_j} products of at most K_{j+1} values and of the factors of Lemma 1, so $b_{j+1} \le E_j + K_{j+1} b_j + O(\sum_\rho (m_\rho + 1))$, with a constant depending on ℓ. Hence b_ℓ is at most a constant times $b_0 + \Phi + 1$, and all values are dyadic with polynomially many bits. □

A chain also connects to OpenAI's partitions. Write a node of the first network as a string of 2^ℓ bits. A row or a column keeps one half of these bits, so a chain selects one bit position, and the two nodes of the last network are the two groups of nodes with that bit fixed; compare the partitions in [OAI26, Sections 2.4 and 2.7], which are into small groups of variables. The sum over a group is not an enumeration here: the proof above evaluates it as nested sums, row by row. On one nested call path, the active summation indices have at most $\gamma^\ell \Phi + \sum_j E_j \le 4\gamma^\ell \Phi$ bits.

For comparison, enumerating the bits of all wires costs $2^{\#\text{wires}}$, and the number of wires can be close to $\Phi/2$. The theorem replaces this exponent by $4\gamma^\ell \Phi$, at the cost of a factor $2^{K_0^2/3}$ that does not depend on the wires.

6 Placing a formula

Theorem 2 helps when $\Phi = O(n)$. This holds if a formula has $O(n)$ occurrences and they are spread evenly over the pairs of nodes. A clause is a set of literals without a variable and its negation, and a formula is a set of clauses. An *occurrence* is a pair (i, C) of a variable i and a clause C containing it.

Lemma 4. *Let $\ell \ge 0$, $K = 2^{2^\ell}$, $\Delta \ge 3$. Let Ψ be a formula with at most three literals per clause, on n variables, each occurring at most Δ times. Then the number of assignments of the variables that occur, which satisfy Ψ, is computed in time $2^{16\Delta\gamma^\ell n}$ times a constant depending on ℓ and Δ, times a polynomial in $n + \Delta$. It is positive if and only if Ψ has a solution.*

Proof. There are at most Δn occurrences. *Placement.* We want few occurrences on each pair of nodes. Number the variables $0, \dots, n-1$, and put them on the nodes cyclically: $\omega(i)$ is i modulo K. A node holds at most $(n+K)/K$ variables, hence at most $\Delta(n+K)/K$ occurrences of its variables. Go through the clauses in any order, and let $\mathrm{load}(s,t)$ count the occurrences (i, C) placed so far with $\omega(i) = s$ and $\omega(C) = t$. Put the clause C on the node t that minimizes $\sum_{i \in C} \mathrm{load}(\omega(i), t)$. For each s, $\sum_t \mathrm{load}(s,t) \le \Delta(n+K)/K$, and C has at most three variables. So the minimized quantity, summed over all K nodes t, is at most $3\Delta(n+K)/K$, and its minimum is at most $3\Delta(n+K)/K^2$. Placing C raises a load by at most $3 \le \Delta$. So all loads stay at most $\kappa = \Delta(3n + 4K^2)/K^2$, using $K \le K^2$.

Network. Each occurrence $o = (i, C)$ is a wire between $\omega(i)$ and $\omega(C)$, with one bit θ_o. Number the wires between two nodes, so that θ_o is one bit of $B(\omega(i), \omega(C))$. The node function F_s is the product of two kinds of tests. For a variable i with $\omega(i) = s$, the test that θ_o is the same for all its occurrences o. For a clause C with $\omega(C) = s$, the test that C is true when each literal reads the bit of its occurrence. Then $\mathrm{val}(K, w, F)$ is the number in the statement.

Potential. We have $w(s,t) \le \mathrm{load}(s,t) + \mathrm{load}(t,s) \le 2\kappa$. So $\sum_{s,t} w(s,t)^2 \le 2\kappa \sum_{s,t} w(s,t) \le 2\kappa \cdot 2\Delta n$, and

$$\Phi(K, w)^2 \le K^2 \cdot 2\kappa \cdot 2\Delta n = 4\Delta^2 n(3n + 4K^2) \le \big(4\Delta(n + K^2)\big)^2.$$

Theorem 2 gives time $2^{K^2/3 + 16\Delta\gamma^\ell(n+K^2)}$ times a constant and a polynomial, and the terms with K^2 are a constant depending on ℓ and Δ. The node functions take polynomial time. □

7 Few occurrences, and the end of the proof

A 3-CNF can have about n^3 clauses, and Lemma 4 needs few occurrences per variable. Branching provides this. It is the sparsification of [IPZ01], in a short form. A clause is a *unit*, *binary* or *ternary* if it has 1, 2 or 3 literals. To *normalize* a formula, delete every clause that properly contains another clause; this keeps the set of solutions.

Lemma 5. *Let $q \geq 2$, and let Ψ be a formula on n variables in which every clause is ternary. There is a binary tree with at most $\mathrm{e}^{(20\ln q+10)n/q}$ leaves, computed in polynomial time per vertex, whose leaves are formulas such that the solutions of Ψ are the union of the solutions of the leaves, and in each leaf a variable occurs at most $4q^2$ times.*

Proof. The root is Ψ. At a vertex, take the first case that applies, with ties broken lexicographically: (i) a literal lies in at least q binaries; (ii) a pair of literals lies in at least q ternaries; (iii) a literal lies in at least q^2 ternaries. If no case applies, the vertex is a leaf. Otherwise let Z be that literal or pair, and write all those clauses as $Z \vee Y_1, \ldots, Z \vee Y_J$. Since $\bigwedge_i (Z \vee Y_i)$ is equivalent to $Z \vee \bigwedge_i Y_i$, every solution satisfies the clause Z or all clauses Y_i. So the vertex has two children, whose solutions together are those of the vertex: the *first child* adds the clause Z, the *second child* adds all clauses Y_i, and both are normalized. At a leaf a literal lies in at most $1 + (q-1) + (q^2-1)$ clauses, so a variable occurs at most $2(q^2+q-1) \leq 4q^2$ times.

It remains to count the leaves. For a vertex, let $\mu = 3q \cdot \#\text{units} + \#\text{binaries}$; the root has $\mu = 0$. We will show that $\mu \leq 8nq$ at every vertex, that μ rises by at least 1 from a vertex to its first child, and that it rises by at least q^2 to its second child. This needs two facts about clauses.

Normalization deletes no added clause. An added clause is new and contains no existing clause; otherwise $Z \vee Y_i$ would properly contain an existing clause and would have been deleted. The added Y_i are distinct and of equal size, so none contains another.

At every vertex, every literal lies in fewer than $2q$ binaries. The root has no binaries. Case (i) adds only units. In case (ii), every literal lies in fewer than q binaries, since case (i) does not apply, and the vertex adds at most one binary. In case (iii), the same holds before the addition, and a literal ν lies in as many added Y_i as there are ternaries containing the pair $\{Z, \nu\}$, which is fewer than q, since case (ii) does not apply.

Measure. There are at most $2n$ units, and fewer than $2nq$ binaries by the bound above. So $\mu \leq 8nq$. A new unit adds $3q$ to μ and deletes fewer than $2q$ binaries, so J new units raise μ by at least $J(q+1)$. A new binary adds 1 and deletes only ternaries. Hence the first child raises μ by at least 1. The second child raises μ by at least q^2: in cases (i) and (ii) it adds $J \geq q$ units, which raise μ by at least $q(q+1)$, and in case (iii) it adds $J \geq q^2$ binaries.

Leaves. So a path from the root has at most $M = 8nq$ edges, and at most $M/q^2 = 8n/q$ of them lead to a second child. Two leaves have different sets of positions of second children, because at the vertex where their paths part exactly one takes the second child. So the number of leaves is at most

$$\sum_{i \leq M/q^2} \binom{M}{i} \leq q^{2M/q^2} \sum_i \binom{M}{i} q^{-2i} = q^{2M/q^2}(1+q^{-2})^M \leq q^{2M/q^2}\mathrm{e}^{M/q^2} = \mathrm{e}^{(16\ln q+8)n/q}.$$

This is at most the bound in the statement, because $16\ln q + 8 \leq 20\ln q + 10$; we state the weaker bound because Lean proves that one. A formula on a vertex has polynomially many clauses, because ternary clauses are only deleted and $\mu \leq 8nq$. So each vertex takes polynomial time. □

Proof of Theorem 1. The parameters depend on ε only. Let $q \geq 2$ be the least integer with $(20 \ln q + 10)/(q \ln 2) \leq \varepsilon/3$. Let ℓ be the least integer with $64q^2\gamma^\ell \leq 2\varepsilon/3$. These exist because the left sides tend to 0. Put $\Delta = 4q^2$.

The algorithm rejects an input with an empty clause, and deletes tautologies and repeated literals. Using two new variables h, h', it replaces a binary clause Y by $Y \vee h$, $Y \vee \neg h$, and a unit clause Y by the four clauses $Y \vee h^{\pm} \vee h'^{\pm}$. This keeps satisfiability and raises n by at most 2, which changes the final bound by the constant factor $2^{2\varepsilon}$. Then it traverses the tree of Lemma 5, and it uses Lemma 4 at each leaf. It accepts if and only if some leaf has a solution. This is correct because the solutions of the formula are the union of those of the leaves.

The tree has at most $\mathrm{e}^{(20 \ln q + 10)n/q} \leq 2^{\varepsilon n/3}$ leaves, and at most twice as many vertices. Up to a constant and a polynomial factor, one leaf costs at most $2^{16\Delta\gamma^\ell n}$ by Lemma 4, and $16\Delta\gamma^\ell = 64q^2\gamma^\ell \leq 2\varepsilon/3$. So the total time is at most a constant times $2^{\varepsilon n}$ times a polynomial in the input size, since the input declares its n variables, so $n \leq |\varphi|$. □

8 Formalization

The Lean 4 development [MU21, mC20] is the library `UncrossingWires` in the directory `lean/` of the repository `https://github.com/suquark/NotETHSimplified`. It uses the libraries `TransposeBase` and `SatBase` of that project for the word language, helpers for brute force, binomials and fields, the network of a formula, the branching tree and the arithmetic choice of its parameter, and the machine layer, and `FormalizationKit` for resource bounds on the machine. The file `UncrossingWires/Audit.lean` reports that the theorems listed there depend only on the axioms `propext`, `Classical.choice` and `Quot.sound`, or on a subset of them, and the library contains no `sorry`. A meta-program, `UncrossingWires/ClosureCheck.lean`, computes that closure, and it counts no constant with a name containing `OnePigeonhole`, `Chebyshev`, `Fourier`, `fourier`, `Hermite`, `indow`, `Matrix.det`, `.det` or `det_`. Table 2 maps the paper to the Lean declarations.

Paper	Lean declaration	Module
Lemma 1	`Ppol_add`, `seed_sum`, `dLen_le`	`Seed`
Lemma 2	`step_val`	`Step`
Lemma 3	`dLen_cheap_le`, `potential_two`, `potential_min`	`Seed`, `Potential`
Theorem 2	`chain_val`, `chainVal_eq`, `core_spec`	`Chain`, `Pseudo.CoreSpec`
Lemma 4	`load_table_le`, `psi_table_le_all`, `BaseNet.val_base`	`Greedy`, `Base`
Lemma 5	`sparseTree_leaves`, `noSparseRule_variable_bound`	`SatBase`
Theorem 1	`Pseudo.Final.pseudo_main`, `Machine.not_ETH`	`Pseudo.Final`, `Machine.FinalMain`

Table 2: Map from the paper to the Lean development. Names are in `UncrossingWires` except those of Lemma 5, in `SatBase`.

The Lean proof departs from the text in these ways. The node functions are integers, with a fixed power of two for each rectangle. Each variable has one more wire, to a node of its own, because Lean counts the assignments of all n variables, and the text those of the variables that occur. A choice is encoded in K'^4 bits with a validity test, not in $K'^4/4$ bits. The potential bound of Lemma 4 is `psi_table_le_all`, for every n. The program instead enumerates small n at the root, for $n \leq K^2$, and uses the sharper bound `psi_table_le` for larger n; the text has no enumeration. Clauses are sets of literals that may hold both signs of a variable, and clauses of fewer than three literals are allowed, with no padding, no new variables and no

deletion of tautologies. The tree of Lemma 5 is the same, and Lean proves its statement with another measure, in `sparseTree_leaves` and `noSparseRule_variable_bound`. The constants of Theorem 1 are computed by hand here, and Lean has its own: `exists_rate` uses a rate $(49/50)^H$, with $(49/50)^2 \geq 24/25$.

The algorithm. The word language has addition, subtraction and multiplication modulo 2^{w}, division, remainder, powers of 2, comparison, and bounded loops. Its semantics gives the value of a program and the number of word operations, counted over the iterations that actually run. The theorem `Pseudo.Final.pseudo_main` states: for every $\varepsilon > 0$ there are one program P and constants C, b, c such that for every n and every formula F with n variables and clauses of at most three literals, with $\mathsf{w} = c(n+1)^3$ and input the bitset of F and the number n, the value of P is nonzero exactly when F is satisfiable, and the number of word operations is at most $C(n+1)^b 2^{\varepsilon n}$. The first-level tables are bound as words; a later level has a constant number of nodes, and its weights are recomputed by nested expressions from the code of the chain. The theorem `Pseudo.core_spec` gives the value and the cost of one network.

The machine. The compiler `compile_correct` of `TransposeBase` compiles every program of this language to a program on Boolean stacks, with a time of at most $7000(\mathsf{w}+1)^2$ source instructions for each word operation, and a bound of $7100(\mathsf{w}+1)^2$ bits on every stack at every prefix. Together with a parser of the input, and a compiler to a primitive multitape machine of OpenAI [OAI26b], whose code is included in `lean/third_party/` with its license and a patch record, this gives `Machine.machine_main`: for every $\varepsilon > 0$ there are a primitive machine and constants such that, for every n and every formula φ with n variables and clauses of at most three literals, the machine started on the fixed encoding of φ halts with the answer, in at most $C(|\varphi|+1)^d 2^{\varepsilon n}$ steps, where $|\varphi|$ is the length of the encoding; and every prefix of that same run visits at most $C'(|\varphi|+1)^{d'}$ tape cells. The word width there is $\mathsf{w} = c(|\varphi|+1)^4 + |\varphi|$. The theorem `Machine.not_ETH` states the failure of ETH in this model. These theorems have the form that refuting ETH requires: a machine for each ε. One algorithm with running time $2^{o(n)}$ is a stronger statement, which Lean does not state. The file `UncrossingWires/Audit.lean` checks by `rfl` that these end statements coincide with those of `TransposeBase`.

What is not formalized. Theorem 2 is stated in Lean only as `Pseudo.core_spec`, for placed networks with one bit per wire, in word operations, with its exponent as an unsummed sum of the E_j and $\gamma^{\ell}\Phi$. The value part, `chain_val`, holds for arbitrary symmetric weights. The proof of Lemma 5 with the measure μ and the constants of this text are not formalized. The script `checks/check_step.py` in the directory of this paper tests the identity and the expansions, the pieces with their coefficients, Lemma 2 in Lean's integer form, and the single-rectangle inequality of Lemma 3, exactly, on tiny networks, including two crossing rectangles for $K' = 3$.

Use of AI

The argument and the Lean proofs were developed in interactive sessions with Claude (Anthropic), building on the public manuscripts [OAI26, O'D26, Cla26, Kin26]. The author is responsible for any errors.

References

[Cla26] Claude (Anthropic). A $2^{o(n)}$ time algorithm for 3-SAT. Preprint, October 26, 2026. https://github.com/anthropics/formal-math/tree/16a5ce41304fcdaea4e0be7c132e1c361dc26da5/eth.

[ETH26] R. Impagliazzo, R. Paturi, and R. Williams. ETH refutation: examination and discussion. Web site, October 2026. https://ethdiscussion.blog/2026/10/26/eth-refutation-examination-and-discussion/.

[IP01] R. Impagliazzo and R. Paturi. On the complexity of k-SAT. *J. Comput. System Sci.* 62(2):367–375, 2001.

[IPZ01] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *J. Comput. System Sci.* 63(4):512–530, 2001.

[Kin26] S. Kintali. A simplified proof of ETH is false. Manuscript, October 26, 2026. https://shivakintali.github.io/papers/ETH.pdf.

[MU21] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proc. 28th CADE*, 2021.

[mC20] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN CPP*, 2020.

[OAI26] OpenAI. The Exponential Time Hypothesis is false. Manuscript, October 17, 2026; revised October 25, 2026. https://github.com/openai/math/blob/42907c9e999cc00d6abe7a50bfc1023250edd1b9/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf.

[OAI26b] OpenAI. Lean formalization accompanying "The Exponential Time Hypothesis is false". Repository, commit 4eaee87; see `lean/OAI/Algorithms/SubexponentialSAT/SqrtLogSpace/CompileSpace.lean`. https://github.com/openai/math/tree/4eaee87548250b2682affda7d05d947c05554bae/lean.

[O'D26] R. O'Donnell. Some exposition of "The Exponential Time Hypothesis Is False". Manuscript, October 30, 2026. https://www.cs.cmu.edu/~odonnell/some_eth_false_exposition.pdf.